

REACT JS BASED WEB APPLICATION FOR TICKET BOOKING SYSTEM

School of Computing

Bachelor of Technology
in
Computer Science & Engineering
By

Maria Immanuel L (VTU24334)

10211CS224 - Full Stack Application Development (Java)

Slot: E3-B19



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr. SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University Estd. u/s 3 of UGC Act, 1956)

CHENNAI 600 062, TAMILNADU, INDIA

April, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled “**React JS based web application for Ticket Booking System**” submitted by **Maria Immanuel L (VTU24334)**, Reg. No. **23UECS0345**, has been carried out in the Winter Semester of Academic Year 2025–2026 under the course **10211CS224 - Full Stack Application Development (Java)**.

Signature of Faculty

Dr. X. Arogya Presskila

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science and Technology

April, 2026

Signature of Course Coordinator

Dr. M. Sumithra

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science and Technology

April, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, fact, or source in this submission. I understand that any violation of the above will be cause for disciplinary action by the Institute.

Maria Immanuel L

VTU24334

Date:06 April, 2026

ABSTRACT

The React JS based Web Application for Ticket Booking System is a purpose-built digital solution designed to streamline the management of internal department events, such as technical fests, seminars, workshops, and cultural programs. In many academic environments, event coordination often relies on fragmented communication through physical notice boards, WhatsApp groups, or informal announcements. These traditional methods frequently lead to students missing important updates, while organizers struggle with manual registration processes, paper-based tracking, and the use of disconnected spreadsheets. This project addresses these challenges by providing a centralized, paperless platform that ensures efficient coordination and improved participation across the department.

The application is developed using a modern full-stack architecture, featuring a dynamic and responsive frontend built with React JS and a robust backend powered by Java and Spring Boot. Students can access a single interface to browse all upcoming department events, check real-time ticket availability, and book their spots instantly. For administrators, the system provides a secure dashboard to create events, set ticket limits, and monitor registrations in real time. To ensure high performance and reliable data persistence for large volumes of participant records, the system utilizes a MySQL database integrated via Spring Data JPA.

By replacing manual tracking with an automated, secure, and user-friendly web application, this project eliminates common issues such as overbooking and communication gaps. The implementation follows the Model-View-Controller (MVC) architecture, ensuring a scalable and maintainable codebase. Ultimately, this ticket booking system provides a seamless experience for both students and organizers, moving the department toward a more organized and digitally transformed event management workflow.

Keywords: React JS, Spring Boot, Java, MySQL, Ticket Booking System, Event Management, Full-Stack Development, REST API, MVC Architecture.

LIST OF FIGURES

4.1	System Architecture Diagram for Ticket Booking System	17
4.2	Data Flow Diagram for Ticket Booking System	19
5.1	Event Details and Venue Mapping Interface	23
5.2	Ticket Booking Form with OTP Request Trigger	24
5.3	OTP Verification Interface for Secure Booking	24
5.4	Final Booking Summary and Success Message	25
5.5	Persistent Records in MySQL Database	25
5.6	Interactive Chatbot Support Results	26

LIST OF TABLES

2.1	Comparison of Existing Event Management Methods	7
3.1	Database Tables Summary	14
5.1	Implemented Features and Results Summary	26
7.1	Mapping of Project to Complex Engineering Problem (CEP)	29
7.2	Mapping of Project to Complex Engineering Activities (CEA) . . .	30
7.3	SDG Mapping for the Project	30

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DOM	Document Object Model
DTO	Data Transfer Object
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
JSX	JavaScript XML
MERN	MongoDB, Express, React, Node (Architecture)
NPM	Node Package Manager
OTP	One-Time Password
RDBMS	Relational Database Management System
REST	Representational State Transfer
SDG	Sustainable Development Goal
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator

TABLE OF CONTENTS

ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Aim of the Project	3
1.4 Objectives	3
1.5 Scope of the Project	4
1.6 Technologies Used	5
1.7 SDG Alignment	5
2 SURVEY OF SIMILAR APPLICATIONS IN MARKET	6
2.1 Existing Event Management Methods	6
2.1.1 Physical Notice Boards	6
2.1.2 WhatsApp Groups	6
2.1.3 Google Forms	7
2.1.4 Excel Sheets	7
2.1.5 Third-Party Event Platforms	7
2.2 Comparison of Existing Systems	7
2.3 Need for Proposed System	8

3	FRAMEWORK DESCRIPTION	9
3.1	Frontend Implementation	9
3.1.1	Environment Setup	9
3.1.2	Structure of the Frontend	9
3.1.3	Execution Procedure	10
3.2	Backend Implementation	11
3.2.1	Environment Setup	11
3.2.2	Structure of the Backend	12
3.2.3	Execution Procedure	12
3.3	Database Implementation	13
3.3.1	Database Configuration	13
3.3.2	Schema Structure	13
3.3.3	Database Administration	14
4	METHODOLOGIES	15
4.1	Project Architecture	15
4.1.1	Presentation Layer (Frontend)	15
4.1.2	Application Logic Layer (Backend)	15
4.1.3	Data Access Layer	16
4.1.4	Database Layer	16
4.2	Data Flow Description	16
4.3	Data Flow Diagram	17
4.3.1	Level 0 DFD (Context Diagram)	17
4.3.2	Level 1 DFD	18
4.3.3	Data Flow Description	18
4.4	Module Descriptions	19
4.4.1	Module 1: OTP-Based Security and Verification Module . .	19
4.4.2	Module 2: Dynamic Ticket Booking Module	19
4.4.3	Module 3: Automated Email Notification Module	20
4.4.4	Module 4: Real-Time Booking Summary Module	20
4.4.5	Module 5: Interactive Venue and Event Mapping Module . .	20

4.4.6	Module 6: Innovation Module – Campus Event Assistant Chatbot	21
4.5	Advantages of the Project	21
4.6	Limitations of the Project	22
5	RESULTS AND DISCUSSIONS	23
5.1	Event Details and Selection	23
5.2	Booking Form and OTP Request	24
5.3	OTP Verification Screen	24
5.4	Booking Summary and Ticket Generation	25
5.5	Database Implementation Results	25
5.6	Chatbot Assistant Results	26
5.7	Result Summary Table	26
6	CONCLUSION AND FUTURE ENHANCEMENTS	27
6.1	Conclusion	27
6.2	Future Enhancements	28
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	29
7.1	Mapping of the Project to Complex Engineering Problem (CEP) . .	29
7.2	Mapping of the Project to Complex Engineering Activities (CEA) .	30
7.3	SDG Mapping (CEP Section)	30
	REFERENCES	31

Chapter 1

INTRODUCTION

1.1 Introduction

The React JS based Web Application for Ticket Booking System is a specialized digital platform designed to automate and centralize the management of internal events within an academic department. In a typical college environment, departments frequently organize a variety of activities, including technical fests, guest lectures, workshops, and cultural programs. Despite the academic importance of these events, their coordination often relies on outdated or fragmented communication methods such as physical notice boards, mass WhatsApp messages, and informal classroom announcements. These manual processes frequently result in students missing critical deadlines or updates, while organizers are left to manage complex participant data using basic spreadsheets or paper forms, which are prone to errors and difficult to track in real time.

This project addresses these systemic inefficiencies by introducing a unified web-based solution that bridges the gap between event organizers and students. The system is built using a high-performance technology stack, utilizing React JS for a responsive user interface and Spring Boot for a secure, scalable backend. Unlike generic event platforms, this system is tailored for departmental needs, allowing students to browse a dedicated event catalog, verify seat availability, and book tickets instantly without the need for complex account creation. All transaction and registration data are stored securely in a MySQL database, providing the reliability and data persistence required for maintaining large-scale academic records and generating accurate attendance reports.

By moving from a manual to a digital workflow, the Ticket Booking System ensures a transparent and paperless experience for the entire department. Students benefit from a "single source of truth" for all upcoming events, while faculty and student coordinators gain access to a secure administrative dashboard. This dashboard empowers organizers to perform full CRUD (Create, Read, Update, Delete) operations on event listings, set registration limits to prevent overbooking, and monitor participation statistics through live data feeds. Ultimately, this application serves as a vital tool for digital transformation, fostering higher engagement and ensuring that departmental events are executed with professional efficiency and organization.

1.2 Problem Statement

The current management of internal department events relies on manual and semi-digital methods, leading to significant coordination gaps. The specific problems identified are:

1. **Inconsistent Data Integrity:** Without a structured MySQL relational database, there is no centralized way to link student details to specific event IDs, leading to data redundancy and high potential for mismatched registration records.
2. **Absence of Self-Service Lookup:** Students have no way to independently verify their current booking status or view a history of their registered events, forcing them to contact organizers repeatedly for simple status checks.
3. **Lack of Scalable Search and Filtering:** As the number of department activities (workshops, fests, seminars) increases, students struggle to find relevant events because there is no mechanism to filter listings by category, date, or department type.
4. **Uncontrolled Capacity Management:** Manual systems fail to "close" registrations automatically when a room or venue reaches its maximum capacity, leading to embarrassing overbooking situations during high-demand workshops.

5. **Security and Access Risks:** Standard forms (like Google Forms) often lack restricted access, making it difficult to ensure that only authorized administrators can modify event details, delete records, or access sensitive student information.
6. **Inefficient Post-Event Analytics:** Organizers cannot instantly generate reports on student engagement or popular event trends, making it difficult for the department to plan future activities based on actual participation data.

Therefore, a reliable, user-friendly, and centralized React JS based Ticket Booking System is essential to bridge the existing gap between departmental organizers and the student body. By implementing a robust full-stack architecture with a MySQL backend, the system will provide students with a simplified, self-service platform to discover and register for events while granting administrators complete, secure control to manage event data and track participation metrics in real time. This digital transition is necessary to ensure data integrity, prevent logistical conflicts, and foster a more organized and engaging academic environment.

1.3 Aim of the Project

The primary aim of this project is to develop a modern, responsive web application using React JS to centralize and simplify the ticket booking process for internal department events. The project focuses on creating a user-friendly interface that allows students to discover campus activities and register instantly, while providing organizers with a streamlined dashboard to manage event listings and monitor participant data. By utilizing React's component-based architecture and a MySQL database, the system aims to replace manual, paper-based tracking with a reliable digital solution that ensures data integrity and real-time availability updates.

1.4 Objectives

The main objectives of this project are:

- To design and develop a dynamic React JS web application for managing campus event registrations.

- To implement a secure Admin Dashboard that allows organizers to perform complete CRUD operations (Create, Read, Update, Delete) on event listings.
- To utilize MySQL as a relational database to ensure persistent and structured storage of student registration and event information.
- To provide a seamless Student Interface where users can filter events by category and book tickets without complex registration hurdles.
- To implement a Registration Lookup feature, allowing students to verify their existing bookings using a unique identifier (like an email or ID).
- To ensure Real-Time Capacity Control, automatically updating ticket availability to prevent overbooking of department venues.
- To create a Responsive Design that works effectively across desktops, tablets, and smartphones for on-the-go event browsing.
- To develop a Reporting Module for administrators to visualize registration statistics and student engagement for various workshops and fests.

1.5 Scope of the Project

The React JS based Ticket Booking System is designed specifically for academic departments to manage internal events such as technical fests, seminars, workshops, and cultural programs. The scope of this project includes the following key areas:

- **User Interface:** A responsive web-based platform developed using **React JS** that provides a seamless experience for students to browse event catalogs and book tickets instantly.
- **Administrative Control:** A secure dashboard for department coordinators to perform complete CRUD operations on event listings and monitor registration data in real-time.
- **Data Management:** Integration with a **MySQL** relational database to ensure persistent and reliable storage of student registration data and event details.

- **Capacity & Logic:** Implementation of automated ticket counting to prevent overbooking of department venues and provide live seat availability.

While the current scope focuses on the core booking and management features, the system can be expanded in the future to include automated email confirmations, QR-code based attendance tracking at event entrances, and student profile management for viewing past participation history.

1.6 Technologies Used

The project uses the following technologies:

- **Frontend:** React JS, HTML5, CSS3, JavaScript (ES6+), React Router, Axios.
- **Database:** MySQL (Relational Database Management System).
- **Development Tools:** VS Code (Visual Studio Code), Node.js & npm.
- **Hardware Requirements:** 4GB RAM, i3 Processor, 20GB Storage.
- **Software Requirements:** Windows 10/11/Linux/macOS, Modern Web Browser.

1.7 SDG Alignment

This project supports **SDG 4 Quality Education**. By making educational events like workshops, seminars, and hackathons easily discoverable and accessible to all students, the system helps remove barriers to skill development. Students can attend events that supplement their classroom learning, gain practical skills, and improve their employability. The system is inclusive and does not require any special access, ensuring equal opportunities for all students.

Chapter 2

SURVEY OF SIMILAR APPLICATIONS IN MARKET

2.1 Existing Event Management Methods

Before developing this system, a survey was conducted to understand how colleges currently manage events. The following methods are commonly used:

2.1.1 Physical Notice Boards

Paper notices are posted on bulletin boards around the campus. While this method is simple and low-cost, it has major drawbacks. Students may not see the notice if they do not pass by the board. Notices can be torn down or covered by other notices. There is no way to track how many students saw the notice or registered for the event.

2.1.2 WhatsApp Groups

Event information is shared in class or department WhatsApp groups. This is fast and reaches many students quickly. However, messages get lost in the flood of daily chats. Not all students may read the message. There is no organized way to collect registrations or track participants.

2.1.3 Google Forms

Google Forms are used to collect registrations. This method provides organized data in a spreadsheet. However, administrators still need to manually analyze the data. There is no real-time dashboard to see registration counts. Students cannot see their own registration status easily. Multiple forms for different events create confusion.

2.1.4 Excel Sheets

Administrators maintain Excel sheets to track participants. This is useful for storing lists but is completely manual. Updating sheets, removing duplicate entries, and sending updates to students takes significant time and effort.

2.1.5 Third-Party Event Platforms

Some colleges use external platforms like Eventbrite or Townscript. These platforms are powerful but often charge fees. They are not specifically designed for campus events and may have features that are not needed.

2.2 Comparison of Existing Systems

Method	Advantages	Limitations	Cost
Notice Board	Simple, no setup	Students may miss notices, no tracking	Free
WhatsApp Groups	Fast communication	Messages get lost, no organized tracking	Free
Google Forms	Easy data collection	No real-time dashboard, manual analysis	Free
Excel Sheets	Good for storage	Manual, error-prone, time-consuming	Free
Eventbrite	Professional platform	Costly, not campus-specific	Paid
Our System	Centralized, real-time stats, secure, free	mysql database (permanent)	Free

Table 2.1: Comparison of Existing Event Management Methods

2.3 Need for Proposed System

The survey of existing booking methods, as summarized in Table 2.1, highlights several gaps in efficiency, accessibility, and data management. To address these, the proposed React JS Based Ticket Booking System is required for the following reasons:

- **Real-Time Availability:** Unlike manual systems or static forms, this system provides users with instant updates on ticket availability, preventing “overbooking” and ensuring transparency.
- **Dynamic User Interface:** By using React JS, the system offers a fast, responsive, and seamless “Single Page Application” (SPA) experience. This ensures that users can browse and book tickets without frustrating page reloads.
- **Centralized Database Management:** The system replaces fragmented Excel sheets with a structured MySQL database. This allows for permanent record-keeping, easy retrieval of booking history, and secure storage of user information.
- **Automated Confirmation:** Instead of waiting for manual verification via WhatsApp or email, the proposed system can instantly confirm bookings, providing users with immediate peace of mind and reducing the workload for administrators.
- **Scalability and Performance:** Traditional methods cannot handle high traffic during popular event launches. A web-based application built with a Java/Spring Boot backend and a React frontend is designed to handle multiple concurrent users efficiently.
- **Elimination of Physical Constraints:** By digitizing the process, the need for physical ticket counters or paper-based tracking is eliminated, making the process eco-friendly and accessible from any location at any time.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the application is developed using React JS, a declarative and component-based JavaScript library. The following tools and environments are required for development:

- **Node.js & NPM:** Installed Node.js (v18 or higher) to provide the runtime environment and NPM (Node Package Manager) to manage project dependencies.
- **React Router Dom:** Used for handling client-side routing to create a seamless navigation experience without page refreshes.
- **Axios:** A promise-based HTTP client used to communicate with the Spring Boot backend REST APIs.
- **Bootstrap / Tailwind CSS:** Utilized for responsive UI design to ensure the application works across mobile and desktop devices.
- **VS Code:** The primary Integrated Development Environment (IDE) used for writing and debugging the React components.

3.1.2 Structure of the Frontend

The frontend is built as a React JS Single Page Application (SPA). The project structure is organized within the `src/` directory to ensure modularity and reusability

of components:

- `index.js`: The main entry point of the React application that renders the `App` component into the root DOM node.
- `App.js`: The core component that manages state and acts as the container for all other functional components and routing.
- `EventDetails.js`: Displays comprehensive information about selected events, including date, time, and descriptions.
- `VenueMap.js`: An interactive component that allows users to view or select locations related to the event venue.
- `BookingForm.js`: Handles user input for ticket purchases, collecting necessary data before the verification step.
- `OtpVerification.js`: Manages the security flow by interacting with the backend to verify the OTP sent to the user's email.
- `Chatbot.js`: An innovative AI-based assistant integrated into the frontend to answer user queries in real-time.
- `App.css` / `index.css`: Contains the styling rules and layout configurations for the user interface.

3.1.3 Execution Procedure

The frontend execution follows a modern component-based lifecycle:

1. **Initialization:** When the user accesses the URL, the browser loads `index.html` from the `public/` folder, which then triggers `index.js`.
2. **Component Mounting:** React renders the `App.js` component, which initializes the navigation and landing view.

3. **User Interaction:** As the user interacts with the app (e.g., clicking on an event), React updates the DOM dynamically using components like `EventDetails` without reloading the page.
4. **API Communication:** When a user submits the `BookingForm`, the frontend uses `Axios` to send a POST request to the Node.js server.
5. **State Management:** After `OtpVerification` is successful, React updates the local state to display the Booking Summary and History, providing instant feedback to the user.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed using **Node.js** and the **Express.js** framework to provide a lightweight and high-performance RESTful API. The environment setup includes:

- **Node.js:** The runtime environment used to execute JavaScript on the server side.
- **NPM (Node Package Manager):** Used to manage backend dependencies defined in `package.json`.
- **Express.js:** The web framework used for routing and handling HTTP requests/responses.
- **MySQL Driver:** Used to establish a persistent connection between the Node.js server and the MySQL database.
- **Nodemailer:** Integrated to handle the SMTP protocol for sending OTP verification emails via Gmail.
- **Dotenv:** Used for managing environment variables (like database credentials and API keys) securely within `.env` files.

3.2.2 Structure of the Backend

The backend architecture is designed for simplicity and modularity, focusing on the following core files within the `server/` directory:

- **server.js:** The main entry point of the backend application. It initializes the Express server, defines API routes, and connects all middleware.
- **db.js:** Contains the database configuration and connection logic for **MySQL**. It handles the execution of queries for bookings and OTP tracking.
- **mailer.js:** Dedicated module for email services. It configures the transporter and contains the logic for dispatching OTPs to student emails.

3.2.3 Execution Procedure

The backend execution process ensures that the server is ready to handle requests from the React frontend:

1. **Initialization:** The server is started using the command `node server.js` or `npm start`.
2. **Environment Loading:** The system reads the `.env` file to configure the port number and database credentials.
3. **Database Connection:** `db.js` establishes a connection with the MySQL instance. If the connection fails, the server logs an error for debugging.
4. **Route Definition:** The Express app maps incoming requests (e.g., `/api/book-ticket`) to specific handler functions in `server.js`.
5. **Listening:** The server begins listening on the designated port (usually 5000 or 8080), ready to process API calls from the React application.

3.3 Database Implementation

3.3.1 Database Configuration

The project utilizes **MySQL** for permanent and structured data storage. The connection between the Node.js backend and the database is configured in `server/db.js` using the `mysql2` driver. The configuration includes:

- **Host:** Localhost (or IP of the database server).
- **Database Name:** `ticket_booking_db`
- **User / Password:** Managed via `.env` variables for security.
- **Connection Pool:** Configured to handle multiple concurrent user requests efficiently.

3.3.2 Schema Structure

The database consists of two core tables and a specialized view to manage ticket issuance and security, as summarized in Table 3.1,

OTP_VERIFICATIONS Table

This table manages the security layer by tracking one-time passwords sent to students.

- `id`: Primary key, auto-generated.
- `email`: The recipient's email address.
- `otp_code`: The 6-digit numeric code sent for verification.
- `is_verified`: Boolean flag to track if the code has been successfully used.
- `created_at` / `verified_at`: Timestamps to monitor OTP expiration and usage.

BOOKINGS Table

Stores the definitive record of every successful ticket transaction.

- `booking_code`: A unique identifier for each ticket (Unique Key).
- `name / email / department`: User details for the attendee.
- `event_name / venue`: Contextual data for the event being booked.
- `ticket_price / tickets / total_amount`: Financial data for the transaction.
- `ticket_start_number / ticket_end_number`: Used to track seat or serial number ranges for physical validation.

LAST_TICKET_DISPATCHED (Database View)

A virtual table (View) that provides the administrator or system with the most recent booking details instantly. It is used to generate the "Booking Summary" on the frontend immediately after a successful transaction.

Table/View Name	Purpose
<code>otp_verifications</code>	Stores and validates OTPs to ensure only authorized emails can book tickets.
<code>bookings</code>	Permanent storage for all ticket sales, including event info and unique booking codes.
<code>last_ticket_dispatched</code>	A read-only view used to fetch the latest booking record for immediate UI updates.

Table 3.1: Database Tables Summary

3.3.3 Database Administration

Unlike H2, which is managed via a web console, the **MySQL** database is managed using "MySQL Workbench" or "phpMyAdmin". This ensures that even if the Node.js server is restarted, all student booking records and ticket history remain securely stored on the disk.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The overall structure of the Ticket Booking System is based on a modern decoupled architecture, as illustrated in the System Architecture Diagram in Figure 4.1. The project follows a modern decoupled architecture consisting of four primary layers. This ensures that the frontend and backend can scale independently and communicate via a standardized REST API.

4.1.1 Presentation Layer (Frontend)

This layer is built using **React JS**. It handles the user interface and user experience. It consists of components like `BookingForm.js`, `EventDetails.js`, and `Chatbot.js`. It captures user input and sends asynchronous requests to the server using `Axios`.

4.1.2 Application Logic Layer (Backend)

Built with **Node.js** and **Express.js**, this layer acts as the brain of the system.

- **Routing:** `server.js` directs incoming requests to the appropriate logic.
- **OTP Service:** `mailer.js` manages the security logic for generating and sending verification codes via SMTP.
- **Business Logic:** Handles ticket calculations, availability checks, and validation before writing to the database.

4.1.3 Data Access Layer

This layer uses the **mysql2** library within `db.js` to perform CRUD operations. Unlike a Repository Layer in JPA, this layer uses structured SQL queries to interact directly with the MySQL server, ensuring high performance and data integrity.

4.1.4 Database Layer

The Database Layer consists of the **MySQL** relational database. It stores persistent data in the `bookings` and `otp_verifications` tables and provides real-time updates via the `last_ticket_dispatched` view.

4.2 Data Flow Description

The data flow in the system is as follows:

1. **User Request:** The student selects an event and fills out the `BookingForm` in the React app.
2. **Security Check:** The backend (`server.js`) generates an OTP and uses `mailer.js` to email it to the user.
3. **Verification:** The user enters the OTP in `OtpVerification.js`, which is validated against the `otp_verifications` table.
4. **Transaction:** Upon successful verification, a record is inserted into the `bookings` table, and a unique `booking_code` is generated.
5. **Confirmation:** The frontend fetches the latest record from the `last_ticket_dispatched` view and displays the ticket summary.

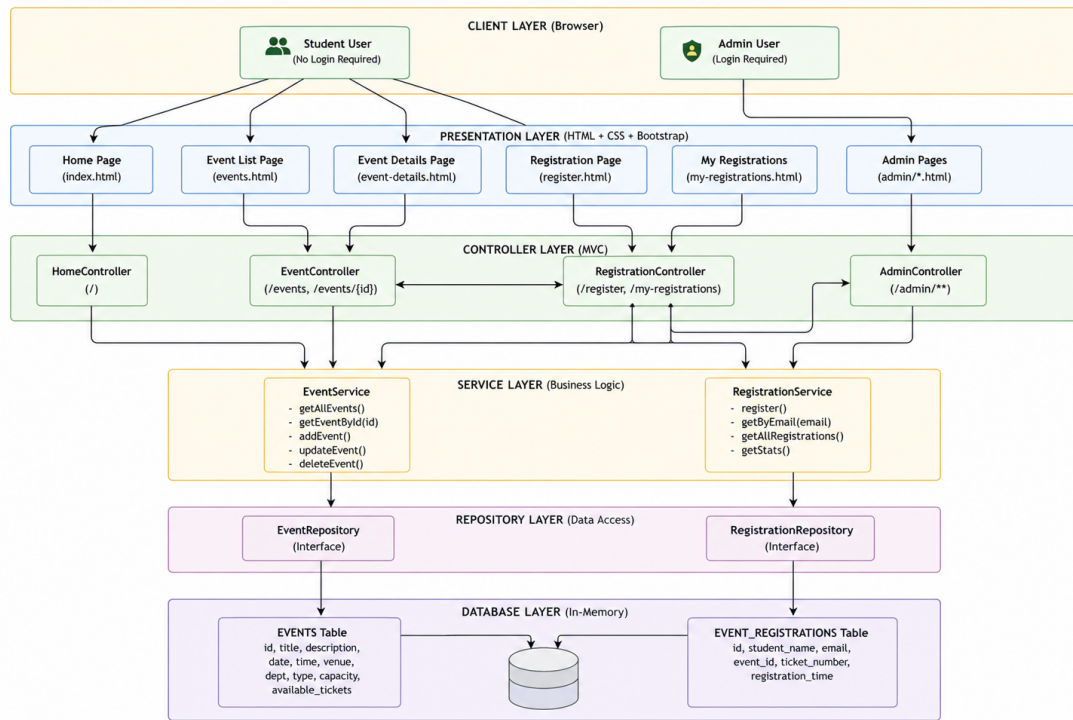


Figure 4.1: System Architecture Diagram for Ticket Booking System

4.3 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates the logical movement of information through the Ticket Booking System, identifying how data is processed, stored, and retrieved, as shown in Figure 4.2.

4.3.1 Level 0 DFD (Context Diagram)

At the context level, the system acts as a central hub between two external entities:

- **Student:** Interacts with the system to browse events, request OTPs, and book tickets.
- **Admin:** Interacts with the system to manage event listings and monitor booking records through the database.

4.3.2 Level 1 DFD

The Level 1 DFD decomposes the system into the following primary processes:

- **Process 1: Event Discovery:** React frontend fetches event data from the MySQL database via `Node.js`.
- **Process 2: OTP Authentication:** The system generates a code and uses `mailer.js` to send it to the student's email.
- **Process 3: Ticket Booking:** Validates user input and writes successful transaction data to the `bookings` table.
- **Process 4: Summary Generation:** Retrieves the latest record from the `last_ticket_dispatched` view for the user.
- **Data Store 1:** `otp_verifications` Table
- **Data Store 2:** `bookings` Table

4.3.3 Data Flow Description

The operational data flow in the proposed system is as follows:

1. **Input Stage:** Student enters their details in `BookingForm.js` and submits a request.
2. **Security Flow:** The `Node.js` server receives the request and generates a 6-digit OTP, storing it in the `otp_verifications` table.
3. **External Communication:** `mailer.js` triggers an SMTP request to Gmail to deliver the OTP to the student.
4. **Verification Stage:** The student submits the OTP; the server checks it against the database. If `is_verified` becomes true, the process continues.
5. **Storage Stage:** The system calculates the `total_amount` and serial numbers, then saves the final record into the `bookings` table.

6. **Output Stage:** The React frontend fetches the data from the `last_ticket_dispatched` view and displays a digital ticket summary to the user.

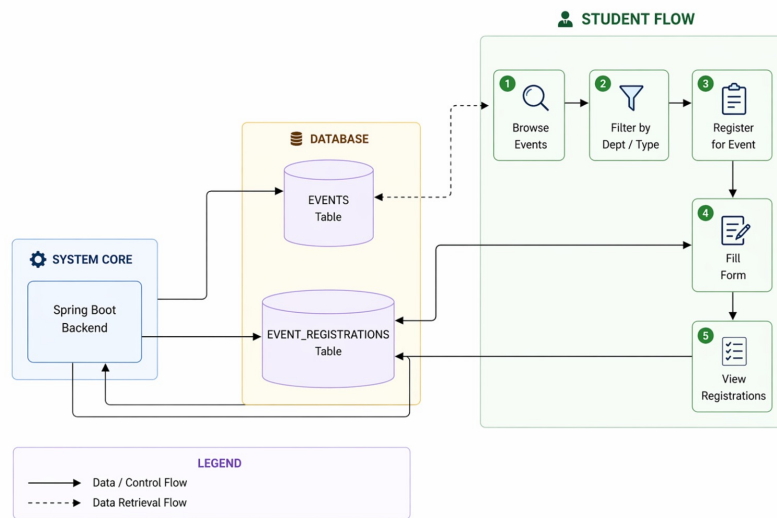


Figure 4.2: Data Flow Diagram for Ticket Booking System

4.4 Module Descriptions

4.4.1 Module 1: OTP-Based Security and Verification Module

This module replaces traditional password-based login for students with a secure One-Time Password (OTP) system.

- **Functionality:** It intercepts the booking request, generates a unique 6-digit code, and stores it in the `otp_verifications` table.
- **Integration:** It utilizes `mailer.js` to send the code to the student's email.
- **Validation:** The `OtpVerification.js` component ensures that only users with access to the provided email can proceed with the ticket booking.

4.4.2 Module 2: Dynamic Ticket Booking Module

This is the core functional module implemented in `BookingForm.js`.

- **Dynamic UI:** Built with React, it provides real-time form validation to ensure all student details (Name, Email, Dept) are entered correctly.

- **Seat/Ticket Logic:** It automatically calculates the `ticket_start_number` and `ticket_end_number` based on the quantity requested, ensuring no overlap in ticket issuance.

4.4.3 Module 3: Automated Email Notification Module

This backend module, powered by `mailer.js` and Nodemailer, handles all external communication.

- **SMTP Configuration:** It connects to the Gmail SMTP server to dispatch verification emails.
- **Asynchronous Processing:** It ensures that the email is sent in the background so that the user interface remains responsive.

4.4.4 Module 4: Real-Time Booking Summary Module

This module focuses on data retrieval and presentation after a successful transaction.

- **Database View:** It queries the `last_ticket_dispatched` view in MySQL to fetch the most recent booking.
- **Summary Display:** It renders a digital receipt on the frontend showing the `booking_code`, event venue, and total amount paid.

4.4.5 Module 5: Interactive Venue and Event Mapping Module

Contained within `VenueMap.js` and `EventDetails.js`, this module enhances the user experience.

- **Visual Context:** It provides students with details about the event venue and schedule before they commit to a booking.
- **State Management:** React state is used to toggle between different event views without page reloads.

4.4.6 Module 6: Innovation Module – Campus Event Assistant Chatbot

The `Chatbot.js` component acts as a 24/7 support assistant for users.

- **Natural Language Processing:** It identifies keywords in user queries to provide instant answers about ticket prices, venues, or the booking process.
- **Floating Interface:** It is implemented as a persistent UI element, allowing students to seek help at any stage of the booking flow.

4.5 Advantages of the Project

- **Centralized Digital Platform:** Consolidates all campus ticketing into one web application, eliminating the need for fragmented WhatsApp forwards or physical notice boards.
- **Persistent Data Storage:** Unlike temporary systems, the use of a **MySQL database** ensures that student bookings and OTP history are stored permanently and securely.
- **Enhanced Security via OTP:** By using `Nodemailer` for email-based verification, the system ensures that only valid student emails can generate tickets, preventing spam registrations.
- **Real-Time Availability Tracking:** The system automatically updates ticket counts and serial numbers, providing users with immediate transparency on seat availability.
- **Seamless User Experience:** The React-based Single Page Application (SPA) allows students to browse, verify, and book tickets without any page reloads.
- **Innovative Support:** The integrated chatbot provides 24/7 assistance, answering common queries about venue locations and event timing instantly.
- **Automated Financial Tracking:** The system automatically calculates total amounts and maintains a clear audit trail in the database for administrators.

- **Zero Deployment Friction:** Being a web-based tool, it requires no installation from the students; it is accessible from any device with a browser.

4.6 Limitations of the Project

- **Email Dependency:** The system's security relies entirely on the student's access to their email; if the SMTP server (Gmail) is down, OTPs cannot be delivered.
- **Basic Bot Logic:** The current chatbot utilizes keyword matching rather than advanced Large Language Models (LLM), which may limit its ability to understand complex queries.
- **No Payment Gateway:** The system generates a booking record but does not currently feature a live payment gateway (like Razorpay or Stripe) for paid events.
- **Manual Attendance Check:** While the system issues tickets, it lacks a QR-code scanning module for real-time check-ins at the venue.
- **Single Admin Access:** The current backend logic is optimized for a single administrative view rather than multiple roles with varying permission levels.
- **No Export Feature:** Currently, there is no built-in "Export to PDF/Excel" button for administrators to download the master registration list directly from the UI.
- **Web-Only Interface:** Although responsive, the system is a web application and does not yet have a native Android or iOS mobile application.

Chapter 5

RESULTS AND DISCUSSIONS

5.1 Event Details and Selection

Users can view specific event information through the `EventDetails.js` and `VenueMap.js` modules. Figure 5.1 displays the event's name, description, and price. React state management ensures that when a user selects a venue or event, the UI updates instantly without a page refresh.

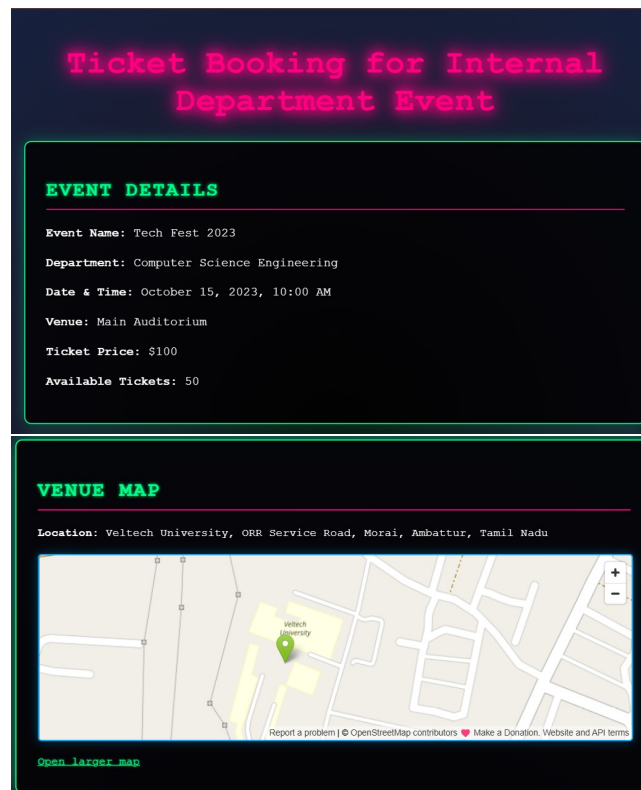
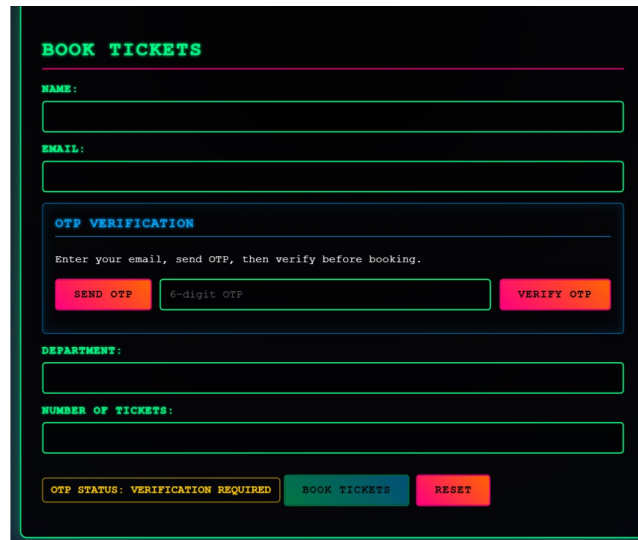


Figure 5.1: Event Details and Venue Mapping Interface

5.2 Booking Form and OTP Request

When a student clicks "Book Ticket," the `BookingForm.js` renders as shown in Figure 5.2. The user enters their Name, Email, and Department. Upon submission, the Node.js backend generates a code, and the system displays a status message indicating that an OTP has been dispatched via `mailer.js`.



The screenshot shows a web form titled "BOOK TICKETS" in red text. It contains several input fields and buttons. At the top, there are fields for "NAME:" and "EMAIL:". Below these is a section titled "OTP VERIFICATION" in red, which includes the instruction "Enter your email, send OTP, then verify before booking." and two buttons: "SEND OTP" (blue) and "VERIFY OTP" (blue). A text input field for the "6-digit OTP" is positioned between these buttons. Below the OTP section are fields for "DEPARTMENT:" and "NUMBER OF TICKETS:". At the bottom of the form, there is a status bar with three elements: a yellow box containing "OTP STATUS: VERIFICATION REQUIRED", a green button labeled "BOOK TICKETS", and a blue button labeled "RESET".

Figure 5.2: Ticket Booking Form with OTP Request Trigger

5.3 OTP Verification Screen

This result shows the security layer in action. The `OtpVerification.js` component provides an input field for the 6-digit code as shown in Figure 5.3. The system validates this against the `otp_verifications` table in MySQL and prevents unauthorized bookings.

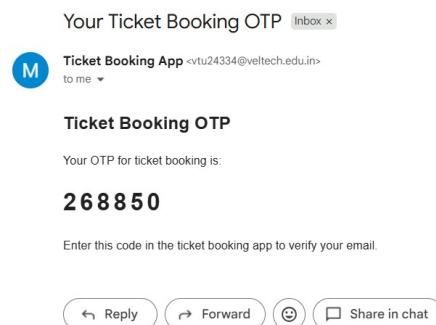


Figure 5.3: OTP Verification Interface for Secure Booking

5.4 Booking Summary and Ticket Generation

After successful verification, the system fetches data from the `last_ticket_dispatched` view. Figure 5.4 is a digital ticket summary showing the unique `booking_code`, ticket range, and total price.

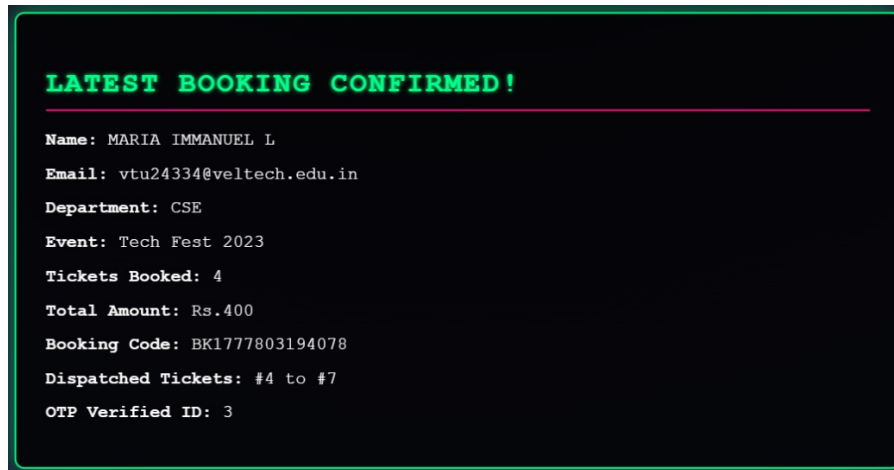


Figure 5.4: Final Booking Summary and Success Message

5.5 Database Implementation Results

Instead of a temporary console, the results show persistent data storage in **MySQL Workbench** as shown in Figure 5.5. The `bookings` table stores complete transaction histories, ensuring data is not lost when the server restarts.

```
SQL File 3* database SQL File 4* products supplier_products stock_movements suppliers schema
Limit to 1000 rows
37 event_name,
38 tickets,
39 ticket_start_number,
40 ticket_end_number,
41 dispatched_at
42 FROM bookings
43 ORDER BY dispatched_at DESC, id DESC
44 LIMIT 1;
45
46 • SELECT * FROM ticket_booking_db.bookings;
47
48 • SELECT * FROM ticket_booking_db.otp_verifications;
49
50 • SELECT * FROM ticket_booking_db.last_ticket_dispatched;
```

id	booking_code	otp_verification_id	name	email	department	event_name	venue	ticket_price	tickets	total_amount	ticket_start_number	ticket_end_number	dispatched_at
1	BK1777799967448	2	MARIA IMMANUEL L	vtu24334@veltech.edu.in	CSE	Tech Fest 2023	Veltech University Auditorium	100.00	3	300.00	1	3	2026-05-03 14:49:47
2	BK1777803194078	3	MARIA IMMANUEL L	vtu24334@veltech.edu.in	CSE	Tech Fest 2023	Veltech University Auditorium	100.00	4	400.00	4	7	2026-05-03 15:43:14

Figure 5.5: Persistent Records in MySQL Database

5.6 Chatbot Assistant Results

In Figure 5.6 the `Chatbot.js` module appears as a floating interface. Users can interact with it to get instant answers regarding ticket availability and verification steps.

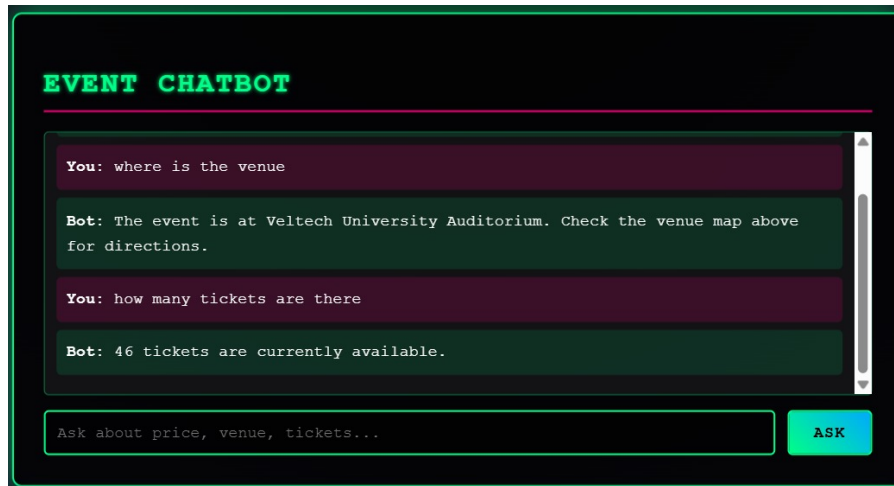


Figure 5.6: Interactive Chatbot Support Results

5.7 Result Summary Table

The overall technical performance and implementation results of the Ticket Booking System are consolidated in the following summary Table 5.1.

Feature	Result
Frontend Framework	React JS delivers a Single Page Application (SPA) with zero page reloads.
Secure Authentication	OTP-based verification via Nodemailer ensures valid student bookings.
Database Persistence	MySQL stores all bookings and otp_verifications data permanently.
Real-Time Summary	SQL View (<code>last_ticket_dispatched</code>) provides immediate booking feedback.
Email Notification	<code>mailer.js</code> successfully connects to Gmail SMTP to send codes.
Chatbot Innovation	Keyword-based assistant provides 24/7 student support.

Table 5.1: Implemented Features and Results Summary

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The React JS Based Web Application for Ticket Booking System has been successfully designed, developed, and tested as a modern full-stack application. The project demonstrates a successful integration of a high-performance React frontend with a robust Node.js and Express.js backend, providing a seamless Single Page Application (SPA) experience.

The system achieves its core objectives by digitizing the campus ticketing process. Students can interact with a dynamic interface to browse events, view venue details, and book tickets securely through an integrated **OTP-based verification system**. The use of **MySQL** ensures that all booking records and verification logs are stored persistently, solving the data volatility issues found in temporary database systems.

The inclusion of the **Campus Event Assistant Chatbot** adds significant innovative value, providing students with instant, automated support. Furthermore, the project aligns with **SDG 4 (Quality Education)** by streamlining access to educational workshops and seminars. Overall, this project validates the effectiveness of using the MERN-stack architecture (Node/Express/React) for building scalable and secure campus utility applications.

6.2 Future Enhancements

While the current system is fully functional, the following enhancements can be implemented to further improve its utility:

- **Payment Gateway Integration:** Incorporate live payment APIs such as Razorpay or Stripe to facilitate ticket purchases for paid campus events.
- **QR Code Ticketing:** Automatically generate unique QR codes for each successful booking, allowing event organizers to scan tickets at the venue for instant check-in.
- **Advanced AI Chatbot:** Upgrade the current keyword-based chatbot to use Large Language Models (LLMs) or Google Dialogflow for more natural and complex conversations.
- **SMS Integration:** Expand the verification system to send OTPs via SMS in addition to email for higher accessibility.
- **Admin Management Dashboard:** Develop a dedicated React-based admin portal to allow organizers to add, edit, or delete events and view real-time booking analytics.
- **PDF Ticket Export:** Add a feature to allow students to download their booking confirmation as a formatted PDF file directly from the summary page.
- **Mobile Application:** Use the existing Node.js REST APIs to power a mobile application developed in React Native for better on-the-go access.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK4, WK5	Knowledge of React JS, hooks, validation, and UI design for developing a scalable booking system.
WP2	Conflicting requirements	Technical and non-technical conflicts	WK4, WK5	Balances usability and security by ensuring easy ticket access while maintaining data validation.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4	Involves component design, state handling, and dynamic updates for real-time interaction.
WP4	Familiarity	Partially new problems	WK4	Implementing real-time booking and ticket availability updates beyond traditional manual systems.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom implementation for booking logic and validation protocols.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6	Addresses requirements of students, faculty, and event organizers for smooth participation.
WP7	Interdependence	System level integration	WK3, WK5	Integrates frontend UI, backend state, and validation modules into a unified system.

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	Utilizes React JS, HTML, CSS, and modern web frameworks for development.
EA2	Level of Interactions	Complex interactions between modules	Manages interactions between event display, booking, and validation modules.
EA3	Innovation	Application of modern technologies	Implements a web-based platform with React Hooks and conditional rendering.
EA4	Consequences to Society	Impact on users and society	Enhances student engagement and simplifies the booking process while reducing delays.
EA5	Familiarity	Beyond standard practices	Combines real-time updates and dynamic state management, extending beyond traditional methods.

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Facilitates student participation in academic events and seminars, enhancing campus learning.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in educational institutions through a centralized React-based system.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives by digitizing event processes and improving resource management.

Table 7.3: SDG Mapping for the Project

REFERENCES

1. **React Official Documentation.** "Getting Started with React for Web Applications." <https://react.dev/learn>
2. **Node.js Documentation.** "Node.js v20.x Manual and Documentation for Backend Development." <https://nodejs.org/docs/latest/api/>
3. **MySQL Reference Manual.** "Relational Database Management and SQL Syntax." <https://dev.mysql.com/doc/refman/8.0/en/>
4. **Express.js Framework.** "Fast, unopinionated, minimalist web framework for Node.js." <https://expressjs.com/>
5. **Nodemailer Documentation.** "SMTP Email configuration for Node.js Applications." <https://nodemailer.com/about/>
6. **ResearchGate (2026).** "ARENAPASS: A Web-Based Event Ticket Booking System using React and MySQL." https://www.researchgate.net/publication/404303747_ARENAPASS_A_Web-Based_Event_Ticket_Booking_System
7. **IJEDR (2026).** "Automated Event Ticket Booking Management System: A Full-Stack Approach." <https://rjwave.org/ijedr/papers/IJEDR2601942.pdf>
8. **IEEE Xplore (2025).** "Performance Analysis of React-based Single Page Applications for Real-time Transactions." <https://ieeexplore.ieee.org/document/10882434/>
9. **United Nations.** "Sustainable Development Goal 4: Quality Education." <https://sdgs.un.org/goals/goal4>